

FCM I Programming Assignment 1

Charles Ohanian

September 16, 2024

1 Executive Summary

In this report, we consider the development and testing of three algorithms for computing a matrix-vector product and one algorithm for computing a matrix-matrix product, assuming triangular matrices. The algorithms differ in the data structures of the given inputs. Randomly generated matrices and vectors are passed through the algorithms and compared to a calculated true result. The error is found to be 0 for all algorithms unless the matrix or the entries within are very large, where we then see machine overflow error.

2 Statement of the Problem

When computing matrix-vector or matrix-matrix products, it is important to consider the structure of the matrix in order to minimize storage. For example, if the matrix is triangular, the 0 entries above or below the diagonal do not have to be stored. If the matrix is unit triangular, the 1's on the diagonal do not have to be stored either. Different data structures can be utilized for the storage of the nonzero entries. In the first two matrix-vector routines, the matrix L is a unit lower triangular matrix stored in either a 2D array or a 1D array using compressed rows. The third matrix-vector routine assumes that L is a banded unit lower triangular matrix with only two subdiagonals, the entries of which should be the only entries stored. The fourth routine, which computes a matrix-matrix product given a unit lower triangular matrix L and an upper triangular matrix U , assumes that L and U are stored together in a single 2D array, as none of their nontrivial entries overlap due to their respective structures. This report analyzes the usefulness of storing triangular matrices in these formats and evaluates the algorithms developed to compute products given these data structures.

3 Description of the Mathematics

3.1 Dense Matrix-Vector Products

A matrix-vector product where the matrix is dense can be computed by taking a series of inner products. Given a square matrix A ,

$$A^T = \begin{bmatrix} \vdots & \vdots & & \vdots \\ r_1 & r_2 & \dots & r_n \\ \vdots & \vdots & & \vdots \end{bmatrix} \in \mathbb{R}^{n \times n},$$

where $r_j \in \mathbb{R}^n$ is the j th column of A . Given a vector $x \in \mathbb{R}^n$, the matrix-vector product is

$$Ax = \begin{bmatrix} r_1^T \cdot x \\ r_2^T \cdot x \\ \vdots \\ r_n^T \cdot x \end{bmatrix} \in \mathbb{R}^n.$$

3.2 Unit Lower Triangular Matrices

If we now restrict the matrix (call it L) to be unit lower triangular, we no longer need to worry about the entries above the diagonal. It is only necessary to iterate through the λ_{ij} 's where $i > j$.

$$L = \begin{bmatrix} 1 & 0 & \dots & 0 \\ \lambda_{21} & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \lambda_{n1} & \lambda_{n2} & \dots & 1 \end{bmatrix}$$

Thus, to compute the matrix-vector product $w = Lv$, where

$$v = \begin{bmatrix} \nu_1 \\ \nu_2 \\ \vdots \\ \nu_n \end{bmatrix} \quad \text{and} \quad w = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \vdots \\ \omega_n \end{bmatrix},$$

$\omega_1 = \nu_1$, $\omega_2 = \nu_2 + \nu_1 \lambda_{21}$, and generally, for $j \in \{1, \dots, n\}$, $\omega_j = \nu_j + \sum_{k=1}^{j-1} \nu_k \lambda_{jk}$. This report will consider two algorithms for computing $w = Lv$ where L is a unit lower triangular matrix, both described in the next section.

3.3 Banded Unit Lower Triangular Matrices

Next, we can consider the case where L is a banded unit lower triangular matrix. That is, L has 1's on the main diagonal and nonzero entries on a finite number of subdiagonals. In this report, we restrict the nonzero entries to the first two subdiagonals ($\lambda_{2,1}, \lambda_{3,2}, \dots, \lambda_{n,n-1}$

and $\lambda_{3,1}, \lambda_{4,2}, \dots, \lambda_{n,n-2}$).

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & \dots & 0 & 0 & 0 \\ \lambda_{21} & 1 & 0 & 0 & 0 & \dots & 0 & 0 & 0 \\ \lambda_{31} & \lambda_{32} & 1 & 0 & 0 & \dots & 0 & 0 & 0 \\ 0 & \lambda_{42} & \lambda_{43} & 1 & 0 & \dots & 0 & 0 & 0 \\ 0 & 0 & \lambda_{53} & \lambda_{54} & 1 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & 0 & \dots & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \dots & \lambda_{n-1,n-2} & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & \dots & \lambda_{n,n-2} & \lambda_{n,n-1} & 1 \end{bmatrix}$$

The process to compute $w = Lv$ remains largely the same as when L was not banded. The only difference is that for a given element of w , ω_j , the summation only iterates from $j - 2$ to $j - 1$ instead of from 1 to $j - 1$. Formally, to compute the matrix-vector product $w = Lv$, where

$$v = \begin{bmatrix} \nu_1 \\ \nu_2 \\ \vdots \\ \nu_n \end{bmatrix} \quad \text{and} \quad w = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \vdots \\ \omega_n \end{bmatrix},$$

$\omega_1 = \nu_1, \omega_2 = \nu_2 + \nu_1 \lambda_{21}$, and for all other entries in w , i.e. for $j \in \{3, \dots, n\}, \omega_j = \nu_j + \sum_{k=j-2}^{j-1} \nu_k \lambda_{jk}$.

3.4 Matrix-Matrix products

Next, we consider the matrix-matrix product $M = LU$ where L is a unit lower triangular matrix and U is an upper triangular matrix. It is assumed that within the combined matrix LU , the entries on the main diagonal and above are the entries from U and the entries below the main diagonal are the entries from L . Denote the combined matrix LU as follows:

$$LU = \begin{bmatrix} \lambda_{11} & \lambda_{12} & \lambda_{13} & \dots & \lambda_{1n} \\ \lambda_{21} & \lambda_{22} & \lambda_{23} & \dots & \lambda_{2n} \\ \lambda_{31} & \lambda_{32} & \lambda_{33} & \dots & \lambda_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \lambda_{n1} & \lambda_{n2} & \lambda_{n3} & \dots & \lambda_{nn} \end{bmatrix}.$$

The process to compute $M = LU$ can be broken down into three parts. When finding M_{ij} :

1. If $i = j$, then $M_{ii} = \lambda_{ii} + \sum_{k=1}^{i-1} \lambda_{ik} \lambda_{ki}$.
2. If $i > j$, then $M_{ij} = \sum_{k=1}^j \lambda_{ik} \lambda_{kj}$.
3. If $i < j$, then $M_{ij} = \lambda_{ij} + \sum_{k=1}^{i-1} \lambda_{ik} \lambda_{kj}$.

3.5 Norms

We now consider how to succinctly characterize the size of a vector or matrix. In this report, we will use the Euclidean norm, or 2-norm, for measuring vectors and the Frobenius norm for measuring matrices. The Euclidean norm for vectors is calculated as follows: Given a vector $x \in \mathbb{R}^n$ with elements ξ_i where $i \in \{1, \dots, n\}$,

$$||x||_2 = \sqrt{\sum_{i=1}^n |\xi_i|^2}.$$

The Frobenius norm for matrices is calculated as follows: Given a matrix $A \in \mathbb{R}^{n \times n}$ with elements α_{ij} where $i, j \in \{1, \dots, n\}$,

$$||x||_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^n |\alpha_{ij}|^2}.$$

4 Description of the Algorithm and Implementation

This report discusses and analyzes the following four subroutines:

1. Compute $w \leftarrow Lv$ where L is a unit lower triangular matrix stored in a 2D array.
2. Compute $w \leftarrow Lv$ where L is a unit lower triangular matrix stored in a 1D array using compressed rows.
3. Compute $w \leftarrow Lv$ where L is a banded unit lower triangular matrix stored in no more than $2n$ storage locations.
4. Compute $M = LU$ where L is a unit lower triangular matrix, U is an upper triangular matrix, and L and U are stored together in a single 2D array.

All routines and their subsequent testers were developed and run in Spyder, an open source python environment included within Anaconda.

4.1 Subroutine 1

The algorithm takes L and v as parameters where L is a 2D array and v is a 1D array. First, w is initialized as an $n \times 1$ vector of 0's, where n is the number of rows of L . Then through a for loop, the algorithm iterates through each element of the output vector w . For each $w[i]$, the algorithm first sets $w[i] = v[i]$ and then adds the sumproduct of the elements of v and the elements below the diagonal in the corresponding row of L . Once it has gone through all n elements of w , the algorithm outputs the vector w .

This algorithm saves storage space by not iterating through the entirety of L to compute each $w[i]$. If the algorithm accessed the entirety of L , the computation of each $w[i]$ would take $2n - 1$ operations, as they are inner products. So to compute the full vector w , it would take

$n(2n - 1) = 2n^2 - n$ operations. Instead, since the algorithm only iterates through the first $i - 1$ entries of each row, the computation of each $w[i]$ takes $2i - 2$ operations where $i \leq n$. As i increases, the number of operations increases, but will always be less than $2n - 1$. Thus, the total number of operations is $\sum_{i=1}^n 2i - 2 = \sum_{i=1}^n 2i - \sum_{i=1}^n 2 = n(n + 1) - 2n = n^2 - n$ operations. Therefore, when compared to a standard matrix-vector calculation, this algorithm reduces the number of operations by n^2 .

4.2 Subroutine 2

The algorithm takes L and v as parameters where L and v are both 1D arrays. The parameter L is simply a list of the nonzero elements of the given unit lower triangular matrix stored in row major order. First, w is initialized as an $n \times 1$ vector of 0's, where n is the number of elements in v . The algorithm then iterates over each element of w , using a single index, L_index , to traverse the compressed representation of L . For each $w[i]$, it starts by setting $w[i]$ to $v[i]$, reflecting that $L[i][i] = 1$ in a unit lower triangular matrix. It then adds each $v[i]$ weighted by the elements of L up to the current row, adjusting L_index accordingly. Once all n elements of w are computed, the algorithm returns w .

Similarly to Subroutine 1, this algorithm saves storage space by not iterating through the entirety of L to compute each $w[i]$. As discussed above, this standard method would take $2n^2 - n$ operations. When L is stored in compressed row format, L has $\frac{n(n + 1)}{2} - n = \frac{n^2 - n}{2}$ elements. Ultimately, although the data is being stored in a different format, the number of operations remains the same as Subroutine 1. Thus, we have reduced the number of operations by n^2 .

4.3 Subroutine 3

The algorithm takes L and v as parameters where L and v are both 1D arrays. The parameter L is simply a list of the nonzero elements below the diagonal of the given banded unit lower triangular matrix stored in row major order. First, w is initialized as an $n \times 1$ vector of 0's, where n is the number of elements in v . The first two elements of w are computed directly. $w[0]$ is set to $v[0]$ and $w[1]$ is computed using the first element of L . For each subsequent element $w[i]$ (from index 2 to $n - 1$), the code computes $w[i]$ by summing $v[i]$ and products of v with appropriate elements from L based on the current L_index . The L_index is updated to move through the compressed representation of L . Once all n elements of w are computed, the algorithm returns w .

As with the first two subroutines, this algorithm saves storage space by not iterating through all of L to compute each $w[i]$. Fewer than $2n$ elements of L are stored. Computing the first element of w takes 0 operations, as it is just set to the first element of v . Computing the second element of w takes 2 operations, as it is the second element of v added to a product. Computing each element after the first two takes 4 operations, as it is the sum corresponding element of v and two products. Thus, the total number of operations is

$0 + 2 + 4(n - 2) = 4n - 6$. This algorithm uses $O(n)$ operations, while a standard matrix-vector multiplication would take $O(n^2)$. Therefore, this algorithm saves a great deal of valuable storage space.

4.4 Subroutine 4

The algorithm takes LU as the lone parameter where LU is the single 2D array that stores both the elements of L , a unit lower triangular matrix and of U , an upper triangular matrix. First, M is initialized as an $n \times n$ matrix of 0's where n is the number of rows of LU . The algorithm then iterates through each element $M[i][j]$ of M . The code breaks this into three parts based on whether $i = j$, $i > j$, or $i < j$. For the diagonal elements (where $i = j$), $M[i][j]$ is set to $LU[i][j]$ and then accumulates contributions from the first $i - 1$ products of the i th row and i th column of LU . For the lower triangular elements (where $i > j$), $M[i][j]$ is computed as the sum of the first j products of the i th row and j th column. For the upper triangular elements (where $i < j$), $M[i][j]$ is computed as the sum of the first $i - 1$ products of the i th row and j th column, added to $LU[i][j]$. Finally, the algorithm returns the computed matrix M .

Storing L and U in a single 2D array as opposed to separate 2D arrays exploits the fact that L and U have no overlapping nontrivial entries (entries other than a 0 or a 1). This reduces the storage space from $2n^2$ to n^2 . Standard matrix-matrix product computations require $2n^3 - n^2$ operations. The number of operations necessary in this presented algorithm is strictly less than this. It would likely take about half as many operations since it is only necessary to iterate through entries up to the diagonal of LU to compute each entry of M . It may still be $O(n^3)$, but the coefficient would be smaller if so.

5 Description of the Experimental Design and Results

Rigorous testing was undertaken to ensure the correctness of the algorithms. A separate tester was developed for each algorithm due to the contrasting nature of the storage of their parameters. The testers for the first three algorithms are similar, as these algorithms all compute matrix-vector products. The fourth tester differs slightly, as the algorithm computes a matrix-matrix product.

5.1 Description of Testers for Subroutines 1, 2, and 3

Each of the first three testers ask the user for a lower and upper bound for dimension, number of trials per dimension, and a lower and upper bound for the integer entries. The tester then generates the appropriate random matrices. For example, suppose the user inputs the bounds for dimension as 50 and 100, inputs the number of trials per dimension as 10, and inputs the bounds for integer entries as -100 and 100 . Then the tester will generate 10 50×50 matrices, 10 51×51 matrices, 10 52×52 matrices, all the way up to 10 100×100 matrices. Every matrix will consist of random integer entries between -100 and 100 . Additionally, every matrix is generated along with a random vector of the appropriate dimension. All random

matrices and vectors are generated using the numpy python package. For each matrix-vector pairing, the first three testers do the following:

1. Converts the random vector to a 1D list.
2. Calculates the true matrix-vector product of the randomly generated matrix and vector using numpy and stores the result in a 2D array.
3. Computes the matrix-vector product using the developed routine.
4. Computes the difference vector, of which each element is the difference between the true and computed element in that position.
5. Computes the Euclidean norm of the difference vector. The Euclidean norm was chosen as a succinct way to measure the error with a single scalar.

The tester then calculates the mean error for the trials at each dimension. Finally, the mean errors for each dimension are displayed in a scatter plot with the dimension n as the independent variable.

5.2 Differences between Testers for Subroutines 1, 2, and 3

Although the first three testers are largely the same, they differ slightly because each randomly generated matrix must be altered to a different format to be passed through each algorithm.

For the first tester, each randomly generated unit lower triangular matrix is already in the form of a 2D array so no modification is necessary to pass the matrix into the algorithm.

For the second tester, we must alter the data structure of the randomly generated matrices, as they are 2D arrays and must be converted to 1D arrays. To achieve this, a list is initialized and through nested for loops, the nonzero elements of each matrix are added to the list in row major order. This 1D array, along with the 1D array containing the random vector, is passed through the routine to compute the result.

The third tester is similar to the second in that the data structure of the randomly generated matrices must be converted to 1D arrays. A list is initialized and through nested for loops, only the two nontrivial entries in each row are added to the list in row major order. This 1D array is then passed through the routine, along with the 1D array containing the random vector.

5.3 Description of Tester for Subroutine 4

Since the fourth subroutine computes a matrix-matrix product, the tester is slightly different in design. It starts off the same however: The user is asked for a lower and upper bound for dimension, number of trials per dimension, and a lower and upper bound for the integer entries. The tester then does the following for each trial:

1. Generates a random unit lower triangular matrix L , using numpy.
2. Generates a random upper triangular matrix U , using numpy.
3. Computes the true matrix-matrix product using numpy and stores the result in a 2D array.
4. Constructs the combined matrix LU where the entries on the diagonal and above are from U and the entries below the diagonal are from L .
5. Computes the product $M = LU$ using the developed routine.
6. Computes the difference matrix, of which each element is the difference between the true and computed element in that position.
7. Computes the Frobenius norm of the difference matrix. The Frobenius norm was chosen as a succinct way to measure the error with a single scalar.

The tester then calculates the mean error for the trials at each dimension. Finally, the mean errors for each dimension are displayed in a scatter plot with the dimension n as the independent variable.

5.4 Results

For each of the four subroutines, two experiments are presented in this report:

1. 10 $n \times n$ matrices are passed through the tester for $n \in \{10, \dots, 100\}$. All entries in the matrices are integers between -100 and 100 .
2. 3 $n \times n$ matrices are passed through the tester for $n \in \{1000, \dots, 1010\}$. All entries in the matrices are integers between -100 and 100 .

These experiments were constructed to demonstrate the accuracy of the code. A high number of medium-sized matrices and a limited number of large matrices are passed through the routines to demonstrate that the code works for all levels of dimension.

Figure 1 shows the results of the two experiments for Subroutine 1. For every dimension between 10 and 100, the mean error for the 10 trials was 0. Additionally, for every dimension between 1000 and 1010, the mean error for the 3 trials was 0. Figure 2 shows the results of the two experiments for Subroutine 2. Similar to the first subroutine, for every dimension between 10 and 100, the mean error for the 10 trials was 0. Additionally, for every dimension between 1000 and 1010, the mean error for the 3 trials was 0. Figure 3 shows the results of the two experiments for Subroutine 3. As with the first two routines, for every dimension between 10 and 100, the mean error for the 10 trials was 0. Additionally, for every dimension between 1000 and 1010, the mean error for the 3 trials was 0. Figure 4 shows the results of the two experiments for Subroutine 4. Again, for every dimension between 10 and 100, the mean error for the 10 trials was 0. Additionally, for every dimension between 1000 and 1010, the mean error for the 3 trials was 0.

Throughout all eight experiments, no error was observed. This highlights the accuracy of the three matrix-vector product algorithms and the matrix-matrix product algorithm. Error can be observed if the dimension of the given matrices and vectors is extremely large, but this simply comes down to machine overflow error.

6 Conclusions

This report offers a detailed evaluation of three algorithms for computing matrix-vector products and one algorithm for computing matrix-matrix products, specifically for triangular matrices. Our comprehensive testing has confirmed that all algorithms perform with exceptional accuracy, consistently producing zero error across a range of matrix dimensions and values. The first three algorithms were designed to handle unit lower triangular matrices stored in different formats: 2D arrays, 1D arrays with compressed rows, and banded matrices with limited nonzero entries. The results demonstrate that these methods are effective in terms of both accuracy and computational efficiency, achieving zero error in all trials regardless of the matrix size or entry values. The fourth algorithm, which computes the matrix-matrix product of combined unit lower and upper triangular matrices, also showed no deviation from the true result, affirming its correctness and reliability. The rigorous testing, which involved generating random matrices and vectors and comparing the computed results with true values, has validated the robustness of these algorithms. The consistent zero error across various test cases highlights the reliability of the proposed data structures and computational strategies, indicating that these algorithms are well-suited for accurate matrix computations in practical applications.

Despite these promising results, there are several potential areas for future research. For instance, investigating alternative data structures or numerical precision settings might uncover opportunities for further optimization. Understanding how these algorithms perform under different conditions could enhance their applicability and effectiveness in a broader range of mathematical and computational problems.

7 Figures

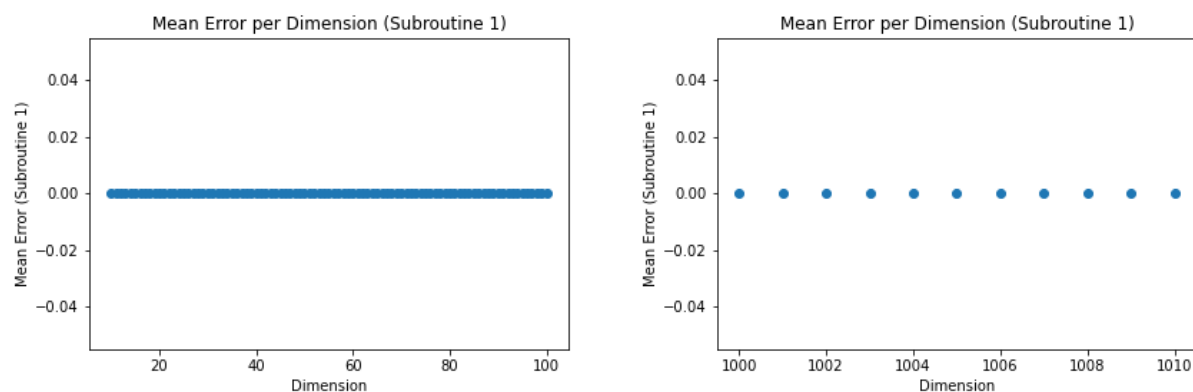


Figure 1. Error results for Subroutine 1. 10 trials per dimension for $10 \leq n \leq 100$ and 3

trials per dimension for $1000 \leq n \leq 1010$. All matrix and vector entries are integers between -100 and 100 .

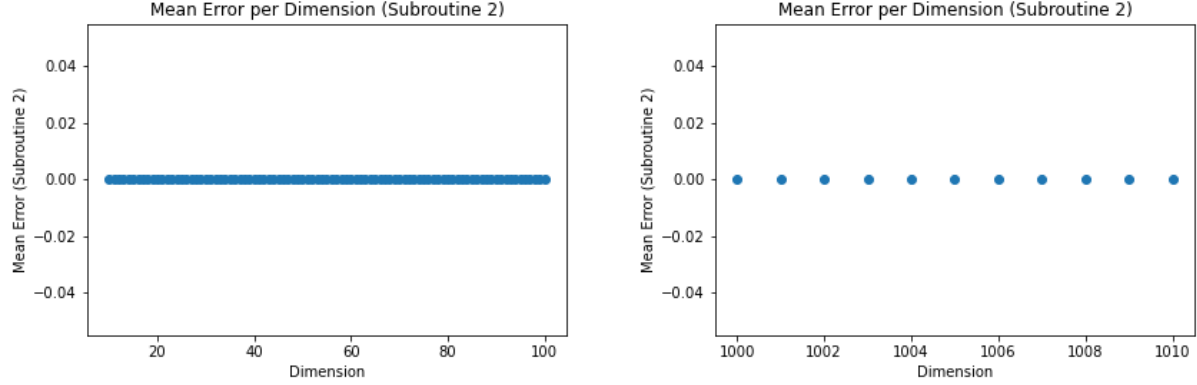


Figure 2. Error results for Subroutine 2. 10 trials per dimension for $10 \leq n \leq 100$ and 3 trials per dimension for $1000 \leq n \leq 1010$. All matrix and vector entries are integers between -100 and 100 .

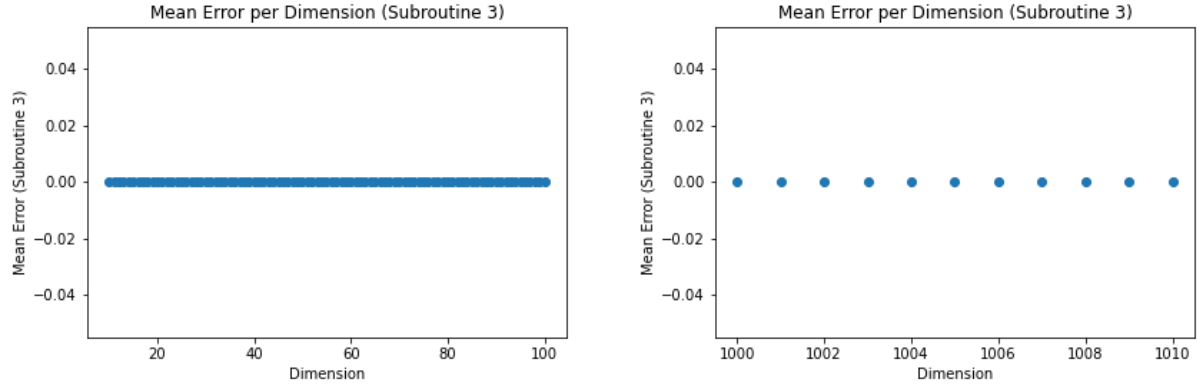


Figure 3. Error results for Subroutine 3. 10 trials per dimension for $10 \leq n \leq 100$ and 3 trials per dimension for $1000 \leq n \leq 1010$. All matrix and vector entries are integers between -100 and 100 .

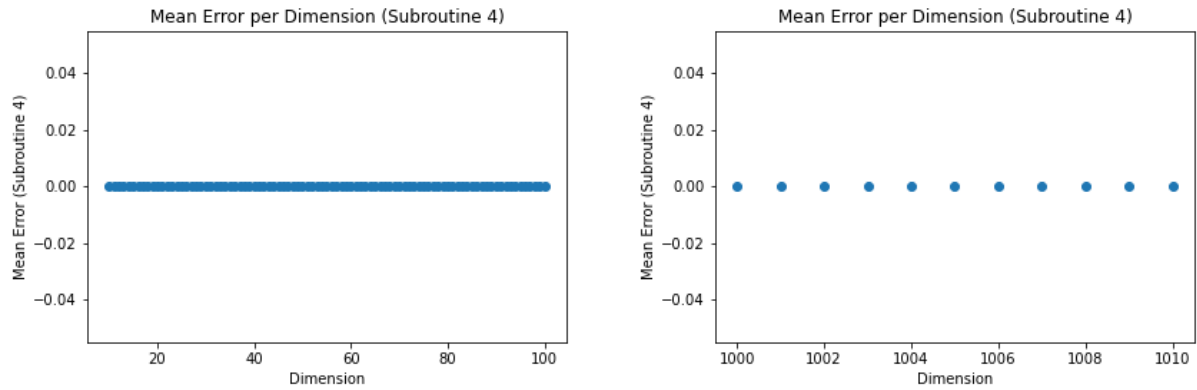


Figure 4. Error results for Subroutine 4. 10 trials per dimension for $10 \leq n \leq 100$ and 3 trials per dimension for $1000 \leq n \leq 1010$. All matrix and vector entries are integers between -100 and 100 .